GUÍA RÁPIDA PARA DEFENSA: RELEVAMIENTO DE OPENCV Y NUMPY

Trabajo Práctico N° 1 (Problema 2: Validación de Planillas) — TUIA · FCEIA · UNR (2026) | Desglose técnico de funciones, parámetros, estructuras de retorno y justificaciones para la defensa oral.

1. Métodos, Constantes y Flags de OpenCV (cv2)

Elemento / Constante	Parámetros y Uso en el Código	¿Qué hace técnicamente?	Tip Clave para la Defensa Oral (¿Por qué se usa así?)
cv2.imread	sheet_path	Carga la imagen de la planilla desde disco como array NumPy de 3 canales en memoria.	**Atención al formato:** Carga en **BGR** (no RGB) con tipo uint8. Se debe convertir antes de operar o graficar.
cv2.cvtColor • **COLOR_BGR2GRAY** • **COLOR_BGR2RGB** • **COLOR_GRAY2BGR**	img, código_conversión	Convierte entre espacios de color: • **BGR2GRAY:** canal de luminancia (grises). • **BGR2RGB:** para mostrar en Matplotlib. • **GRAY2BGR:** 3 canales para pintar recuadros color.	**Justificación:** Grises reduce de 3 canales a 1 para Otsu. Matplotlib espera RGB (si se pasa BGR se ve azul). GRAY2BGR permite dibujar bounding boxes en color sobre máscaras binarias.
cv2.threshold • **THRESH_OTSU** • **THRESH_BINARY_INV**	img_gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU	Binarización global adaptativa. Minimiza la varianza intra-clase (Otsu). Devuelve (umbral, img_th).	**¿Por qué thresh=0?** Otsu calcula el umbral solo; el valor 0 se ignora. **¿Por qué BINARY_INV?** El texto original es oscuro sobre papel claro. En PDI los objetos deben ser blancos (255) y el fondo negro (0).
cv2.connectedComponentsWithStats • **cv2.CV_32S** • **8 (conectividad)**	celda.astype(np.uint8), 8, cv2.CV_32S Retorna: n, labels, stats, centroids	Segmenta y calcula estadísticas morfológicas de cada componente de la celda en un único pase.	**¿Por qué 8-conectividad?** Evita que trazos diagonales de letras se quiebren en piezas separadas. **¿Por qué stats[1:]?** El índice 0 siempre representa el **fondo**, no una letra.
Constantes de stats: • **cv2.CC_STAT_LEFT** • **cv2.CC_STAT_WIDTH** • **cv2.CC_STAT_AREA**	Columnas de indexación de la matriz stats de componentes conexas.	• **LEFT / WIDTH:** posición X y ancho del bounding box. • **AREA:** cantidad de píxeles encendidos del objeto.	**Filtrado y palabras:** • AREA > 2 px: elimina ruido puntual y restos de grilla. • Distancia horizontal (gap): LEFT[i+1] - (LEFT[i] + WIDTH[i]). Si gap > 6 px, es espacio entre palabras.
cv2.findContours • **cv2.RETR_CCOMP** • **cv2.CHAIN_APPROX_SIMPLE**	letra, cv2.RETR_CCOMP, cv2.CHAIN_APPROX_SIMPLE Retorna: contornos, jerarquía	Extrae contornos vectoriales y construye una jerarquía topológica de dos niveles (externo vs agujeros internos).	**Euler y clasificación ('L' vs 'R'/'A'):** • Si jerarquia[0][i][3] != -1, tiene padre (es un hueco interno). • 'L' = 0 agujeros. • 'R' y 'A' = 1 agujero (se desempatan por trazo vertical).
cv2.drawContours	vis, contornos, idx, color, grosor	Dibuja contornos poligonales en una imagen (usado en el gráfico de diagnóstico de Euler).	Permite visualizar en colores distintos el perímetro externo (amarillo) de los agujeros cerrados (rojo).
cv2.resize	crop, (target_w, target_h)	Redimensiona el recorte de Nombre y Apellido para encajarlo de forma estilizada en la tarjeta de no aprobados.	**Ojo al orden:** En OpenCV el tamaño es (ancho, alto); en NumPy la forma es (alto, ancho).
cv2.rectangle	canvas, pt1, pt2, color, thickness	Dibuja tarjetas, fondos de encabezados, insignias (badges) y recuadros de componentes.	**Espesor -1:** Rellena la figura con color sólido (muy usado para fondos de tarjetas y botones de estado).
cv2.line	grid_vis, pt1, pt2, color, thickness	Traza líneas rectas entre dos coordenadas para superponer la rejilla detectada en el diagnóstico.	Verifica visualmente si las proyecciones 1D coincidieron exactamente con las líneas impresas de la grilla.
cv2.putText • **FONT_HERSHEY_SIMPLEX** • **LINE_AA**	canvas, text, org, font, scale, color, thick, LINE_AA	Renderiza texto vectorial sobre las imágenes de diagnóstico y el panel de salida de no aprobados.	**¿Por qué LINE_AA?** Aplica suavizado anti-aliasing (elimina el pixelado serruchado en los bordes de las letras).

2. Desglose en Detalle de Retornos Críticos (OpenCV)

Estructura exacta de datos, dimensiones, significado de cada índice y cómo responder en la defensa oral.

A. Retorno de cv2.connectedComponentsWithStats: (n, labels, stats, centroids)

Variable Retornada	Tipo y Forma (Shape)	Significado y Contenido de los Datos	Uso en el Código y Pregunta de Examen
n **(num_labels)**	int (escalar)	Cantidad total de componentes conexas halladas **+ 1** (porque incluye al fondo como etiqueta 0).	Si en una celda dice 'SOY' (3 letras aisladas), n = 4 (etiqueta 0 fondo, 1, 2 y 3 letras).
labels	ndarray 2D (alto, ancho) tipo: int32	**Mapa de segmentación píxel a píxel.** Cada coordenada (y, x) contiene el ID de la componente a la que pertenece (0 para fondo, 1 a $n-1$ para letras).	En el código se descarta con _ (guion bajo) porque no coloreamos píxel por píxel; nos alcanzan las estadísticas globales de stats.
stats **(La más importante del TP)**	ndarray 2D (n, 5) tipo: int32	**Tabla geométrica de 5 columnas** por componente ($i = 0$ a $n-1$): • [0] CC_STAT_LEFT: $X_{\min}$ del bounding box. • [1] CC_STAT_TOP: $Y_{\min}$ del bounding box. • [2] CC_STAT_WIDTH: Ancho en píxeles del box. • [3] CC_STAT_HEIGHT: Alto en píxeles del box. • [4] CC_STAT_AREA: Píxeles reales encendidos.	**¿Por qué hacemos stats[1:]?** **La fila stats[0] es el fondo (área enorme que cubre toda la celda). Se descarta con [1:] para quedarse exclusivamente con las letras.** **Área > 2 px: elimina ruido.** **Huecos: LEFT[i+1] - (LEFT[i] + WIDTH[i]) > 6 px marca espacio.**
centroids	ndarray 2D (n, 2) tipo: float64	Centro de masa o baricentro geométrico (c_x, c_y) de cada componente, calculado promediando las coordenadas de sus píxeles.	Se descarta con _ en el código porque para ordenar de izquierda a derecha usamos el borde izquierdo (CC_STAT_LEFT).

B. Retorno de cv2.findContours: (contornos, jerarquia)

Variable Retornada	Tipo y Forma (Shape)	Significado y Contenido de los Datos	Uso en el Código y Pregunta de Examen
contornos	tuple[ndarray] Longitud: M Cada uno: (K, 1, 2) tipo: int32	Lista de polígonos frontera. Cada elemento contiene las K coordenadas [x, y] de los vértices del contorno. Con CHAIN_APPROX_SIMPLE solo se almacenan puntos de inflexión (ahorra memoria comprimiendo líneas rectas).	Se usan en cv2.drawContours para dibujar el contorno perimetral externo (amarillo) y las cavidades internas (rojo) en el gráfico diagnóstico.
jerarquia **(hierarchy)**	ndarray 3D (1, M, 4) tipo: int32	Árbol de relaciones topológicas de los M contornos. Para cada contorno i, contiene un vector de 4 enteros: [Next, Previous, First_Child, Parent] • **[0] Next:** Siguiente en mismo nivel (-1 si no hay). • **[1] Previous:** Anterior en mismo nivel (-1 si no hay). • **[2] First_Child:** Primer contorno hijo interior (-1 si no hay). • **[3] Parent:** Contorno padre que lo encierra (-1 si es exterior).	**¿Por qué miramos j[3] != -1?** **Al usar RETR_CCOMP se crea una jerarquía de exactamente 2 niveles: contornos exteriores y agujeros.** **Si Parent != -1 (la posición [3]), el contorno es un agujero cerrado interno.** **• 'L': 0 agujeros (Euler).** **• 'R' y 'A': 1 agujero (desempate por trazo vertical).**

3. Métodos y Funciones de NumPy (numpy / np)

Función / Operación	Uso en el Código y Parámetros	¿Qué hace técnicamente?	Tip Clave para la Defensa Oral (¿Por qué se usa así?)
np.sum (Proyecciones 1D)	• np.sum(img_th, axis=1) (filas) • np.sum(img_th, axis=0) (columnas) • np.sum(huecos > umbral)	Calcula la suma marginal a lo largo de un eje: • **axis=1:** suma horizontal (proyección en Y). • **axis=0:** suma vertical (proyección en X). • Sin eje: suma elementos booleanos.	**¿Por qué proyecciones y no Hough?** Las líneas de la tabla son perfectamente ortogonales. La proyección 1D es lineal en tiempo $\mathcal{O}(N \times M)$, sin costo de acumulación ni falsos positivos por ruido.
np.where	• np.where(proyeccion > umbral)[0] • np.where(np.diff(idx) > 1)[0]	Devuelve los índices de las posiciones donde la condición booleana se cumple como verdadera (True).	Permite aislar exactamente qué filas y columnas contienen líneas negras densas de la tabla, y detectar saltos entre líneas consecutivas.
np.diff	np.diff(idx)	Calcula la diferencia discreta hacia adelante entre elementos contiguos: $\Delta x_i = x_{i+1} - x_i$.	**Agrupamiento de líneas físicas:** Si una línea impresa mide 2 o 3 px de ancho, np.diff(idx) == 1. Cuando np.diff(idx) > 1, se detecta un salto a la *siguiente* línea de la tabla.
np.concatenate	• np.concatenate(([idx[0]], idx[cortes + 1])) • np.concatenate((idx[cortes], [idx[-1]]))	Une arreglos 1D a lo largo del eje especificado para construir los vectores de inicios y fines.	Permite delimitar el intervalo (inicio, fin) de cada línea gruesa de la rejilla para luego indexar las celdas limpiamente.
np.argsort	stats[np.argsort(stats[:, cv2.CC_STAT_LEFT])]	Devuelve los índices que ordenarían el array de menor a mayor según la columna de posición X izquierda.	**Orden natural de lectura:** OpenCV devuelve las componentes conexas en orden arbitrario. Con argsort las ordenamos estrictamente de izquierda a derecha.
np.ones / np.zeros	• np.ones((h, w, 3), dtype=np.uint8) * 250 • np.zeros((h, w, 3), dtype=np.uint8)	Crea matrices inicializadas con unos o ceros con el tamaño y tipo de dato especificados.	• np.ones(...) * 250 genera un lienzo blanco/gris claro limpio para renderizar el panel de no aprobados. • np.zeros(...) genera máscaras negras vacías para trazar contornos.
np.uint8	• celda.astype(np.uint8) • dtype=np.uint8	Tipo entero sin signo de 8 bits (rango 0 a 255). Es el tipo nativo exigido por casi todas las funciones de OpenCV.	Si se le pasa un array booleano o float a funciones como findContours o connectedComponents, OpenCV lanza error. uint8 garantiza compatibilidad estricta.

Detalle Anatómico de Retornos en Métodos de NumPy

Función NumPy	Tipo y Forma (Shape)	¿Qué retorna exactamente?	¿Por qué se indexa así en el código?
np.where(...)	tuple[ndarray] Longitud = dimensiones del input.	Una tupla con un array de índices por cada dimensión donde la condición se cumple.	**¿Por qué [0] al final?** Como la proyección es un vector 1D, devuelve (array,). El [0] extrae directamente el array NumPy de índices sin la tupla envolvente.
np.diff(...)	ndarray 1D Longitud: $N - 1$	Array con las diferencias finitas sucesivas: $\text{out}[i] = \text{idx}[i+1] - \text{idx}[i]$.	Permite saber si dos píxeles contiguos pertenecen a la misma línea gruesa ($\Delta = 1$) o si hay un salto a la siguiente línea de la tabla ($\Delta > 1$).
np.argsort(...)	ndarray 1D Longitud: N, enteros	Arreglo con los **índices de permutación** que ordenarían los datos de menor a mayor.	No ordena un vector suelto; nos permite **reordenar filas enteras de la matriz stats** según la columna X (`CC_STAT_LEFT`) en orden de lectura natural.
.any(axis=1).mean()	Paso 1: ndarray (H,) bool Paso 2: float (escalar)	• any(axis=1): True si hay píxel de trazo en esa fila. • mean(): Promedio aritmético (proporción entre 0.0 y 1.0).	**Mide la cobertura vertical del trazo en la franja izquierda. Si cobertura > 0.90, es la columna recta de la letra **'R'**; si no, es la letra **'A'**.**

4. Banco de Preguntas Clave para la Defensa Oral

Preguntas conceptuales y metodológicas típicas de la mesa evaluadora de la cátedra:

1. ¿Por qué no utilizaron una librería de OCR como Tesseract para leer los campos?	Porque el trabajo práctico evalúa explícitamente los fundamentos de las **Unidades 1 a 3** (perfiles de proyección, segmentación, morfología, componentes conexas y Euler). Tesseract sería una 'caja negra' que no demuestra el dominio del análisis de bajo y medio nivel. La solución implementada es completamente determinística, no depende de modelos entrenados externos y cumple estrictamente el programa de la materia.
2. ¿Por qué se utilizó una proyección 1D con np.sum en lugar de la Transformada de Hough para la grilla?	La Transformada de Hough busca líneas en cualquier ángulo en un espacio de acumulación discretizado (ρ, θ), lo que demanda alto costo computacional y requiere post-filtrado para descartar líneas falsas por ruido o texto. Dado que el formulario está alineado y su grilla es ortogonal (horizontal y vertical), las **proyecciones 1D** reducen el problema a dos sumas marginales de complejidad lineal $\mathcal{O}(N \times M)$, siendo matemáticamente exactas, rápidas y robustas.
3. ¿Cómo resuelve el código que las líneas de la grilla impresa tengan más de 1 píxel de espesor?	Con la combinación de np.diff y np.where en la función detectar_lineas. Se agrupan píxeles contiguos cuya proyección supera el umbral. Cuando la diferencia entre índices consecutivos es 1 ($\Delta = 1$), pertenecen a la misma línea gruesa. Al superarse la distancia de 1 px ($\Delta > 1$), se marca el fin de la línea actual y el inicio de la siguiente, evitando contabilizar múltiples líneas por un mismo trazo.
4. ¿Cómo diferencian una palabra de dos palabras sin reconocer caracteres?	A través de los **gaps horizontales** entre componentes conexas ordenadas por su coordenada X izquierda (cv2.CC_STAT_LEFT). La distancia entre dos caracteres consecutivos es $\text{gap} = \text{LEFT}_{i+1} - (\text{LEFT}_i + \text{WIDTH}_i)$. Si ese espacio en blanco supera el umbral empírico calibrado (UMBRAL_ESPACIO = 6 px), se contabiliza como un espacio inter-palabra. Así: $\text{Palabras} = \text{espacios} + 1$.
5. ¿Por qué se utiliza la jerarquía de contornos (findContours) para la Condición Final en lugar de plantillas?	Porque las tres letras posibles ('L', 'R', 'A') tienen propiedades topológicas invariantes a tamaño, traslación y fuente: la 'L' es simplemente conexa y posee **0 agujeros internos** (género de Euler 1), mientras que la 'R' y la 'A' poseen exactamente **1 agujero cerrado**. Al usar cv2.RETR_CCOMP, cualquier contorno con Parent != -1 es un agujero interior. Finalmente, 'R' se diferencia de 'A' porque la 'R' tiene un asta vertical continua a la izquierda (cobertura $> 90\%$).
6. ¿Qué funciones propias componen el pipeline del script y qué devuelven?	• detectar_lineas(proyeccion, umbral): devuelve lista de tuplas (inicio, fin) de cada línea gruesa. • detectar_celdas(img_th): devuelve filas, cols, registros_coords, campos con los límites de corte (y_0, y_1, x_0, x_1) de cada celda. • componentes(celda): devuelve la matriz stats filtrada (sin fondo y con $\text{área} > 2\text{ px}$) y ordenada de izquierda a derecha. • contar(celda): devuelve la tupla (cant_caracteres, cant_palabras).